

JavaAgent-CLI

开发和安装手册

基于 ReAct 架构的自主编程智能体

版本：1.0.0

日期：2026 年 6 月 12 日

开发者（排名不分先后）：莫承潜，黄春云，胡鸿扬

汇报者（排名不分先后）：黄麟松，王郅为

四川农业大学 · 信息工程学院

本项目采用 **MIT 开源协议** (MIT License) 发布。

版权所有 © 2026 四川农业大学信息工程学院 JavaAgent-CLI 开发团队。

在遵守 MIT 协议条款的前提下，任何人都可以自由使用、复制、修改、合并、发布、分发、再许可和/或出售本软件的副本。

目 录

一、项目简介	3
二、系统架构	4
三、环境要求	5
四、安装与构建	5
五、配置说明	6
六、使用指南	7
七、内置工具	8
八、斜杠命令	9
九、安全机制	10
十、插件扩展	11
十一、项目结构	12
十二、常见问题	13

一、项目简介

1.1 什么是 JavaAgent-CLI

JavaAgent-CLI 是一个用 Java 21 实现的自主编程智能体，类似于 Claude Code 的命令行版本。它采用 **ReAct (Reasoning + Acting)** 架构，让大语言模型通过工具调用与本地开发环境交互，自动完成编程任务。

用户输入自然语言指令后，智能体自主规划、推理、调用工具（文件编辑、搜索、Shell 执行等）来完成任务，整个过程在终端中实时可见。

1.2 核心特性

特性	说明
ReAct 推理循环	观察-推理-行动-反思，每轮最多 12 次迭代
8 个内置工具	read_file、write_file、edit、delete_file、grep、list_directory、bash、network
SSE 流式输出	逐 token 实时显示 AI 回复，支持思维链推理
人机协作审批	只读操作自动批准，写入操作需用户确认
上下文可视化	/context 命令展示 token 使用分布
5 级推理深度	从"简短直接"到"极限推理"可调
安全沙箱	工作区隔离、危险命令检测、敏感信息脱敏
跨平台支持	macOS / Linux / Windows (PowerShell/cmd)

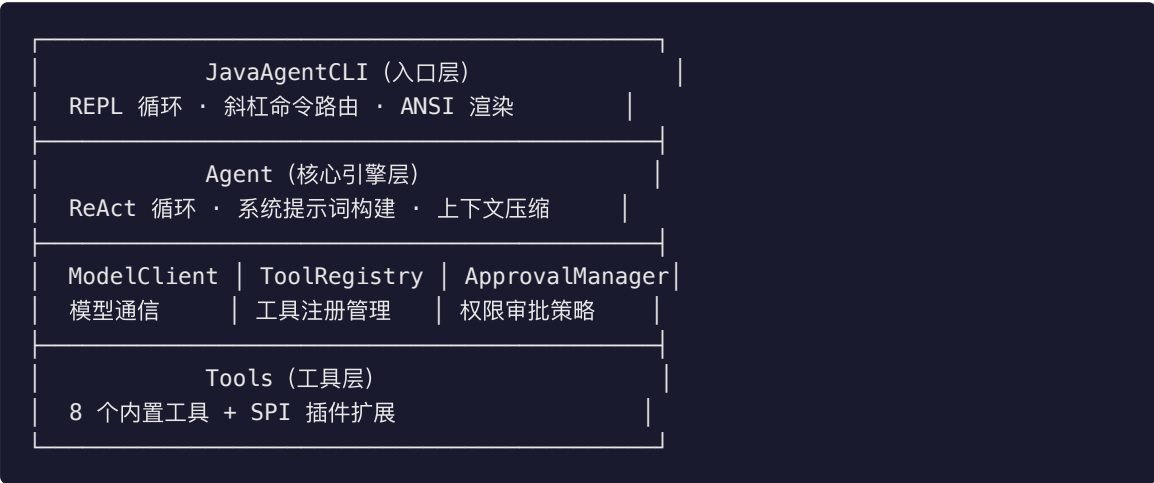
1.3 技术栈

组件	版本	用途
Java	21+	运行环境
Maven	3.8+	构建工具
Jackson	2.18.3	JSON 序列化
JLine 3	3.28.0	终端交互、行编辑、Tab 补全
jtokkit	1.1.0	Token 计数 (cl100k_base)
JUnit 5	5.11.4	单元测试与集成测试

二、系统架构

2.1 整体架构

JavaAgent-CLI 采用分层架构设计，各层职责清晰：



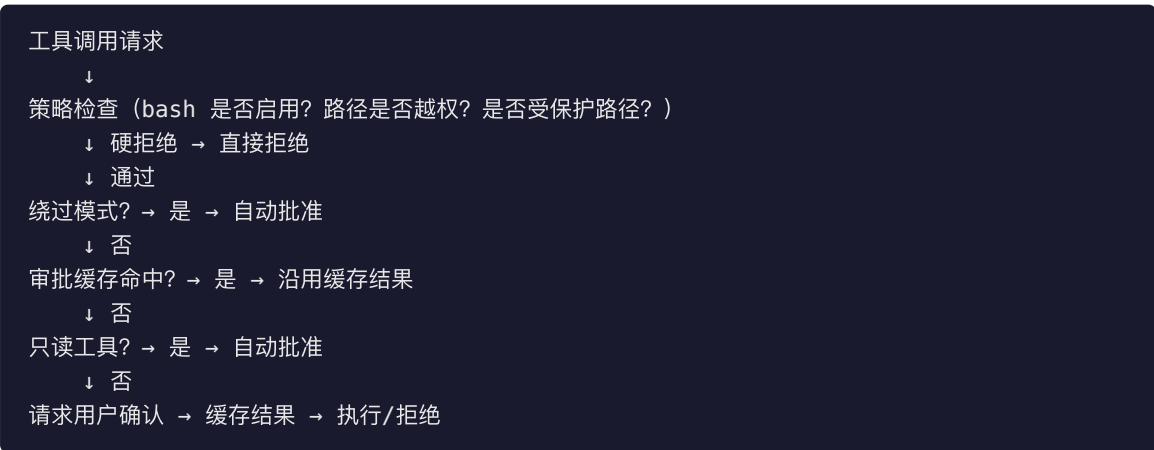
2.2 ReAct 循环流程

每次用户输入后，Agent 执行以下循环：

1. 将用户消息加入对话历史
2. 构建系统提示词（身份 + 规则 + 工具定义）
3. 调用 LLM 获取响应（文本回复 或 工具调用）
4. 如果响应包含工具调用 → 执行工具 → 将结果返回 LLM → 回到第 3 步
5. 如果响应是文本回复 → 返回给用户
6. 达到最大迭代次数（默认 12）→ 停止并提示

2.3 审批流程

工具执行前经过审批检查：



三、环境要求

3.1 必需环境

软件	最低版本	说明
JDK	21	需要支持虚拟线程和模式切换
Maven	3.8	构建工具
操作系统	—	macOS、Linux、Windows 均可

3.2 真实模式额外要求

- 有效的 OpenAI 兼容 API Key（支持 OpenAI、DeepSeek、Moonshot 等）
- 网络连接（用于调用 AI API）

3.3 模拟模式（不推荐）

无需 API Key 和网络，用于开发和演示。使用 `--mock` 参数启动。

四、安装与构建（Tips：手动部署按顺序依次复制即可）

4.0 一键部署（推荐）

如果你使用小龙虾（Lobster）或 Hermes 等 AI 编程助手，只需发送下面这句话即可自动完成项目下载、构建和部署：

```
帮我下载、构建并部署这个项目：https://github.com/cuac333/JavaAgent-CLI
```

AI 将自动克隆仓库、安装依赖、编译项目并完成运行环境配置。

4.1 获取源码

```
git clone https://github.com/cuac333/JavaAgent-CLI.git
```

4.2 构建项目

```
cd JavaAgent-CLI
mvn clean package -DskipTests
```

4.3 运行程序

macOS / Linux

```
# 方式一：使用包装脚本（推荐）
chmod +x javaagentcli
./javaagentcli

# 方式二：直接运行 JAR
java --enable-native-access=ALL-UNNAMED -jar target/javaagent-cli-1.0.0.jar
```

Windows

```
# 方式一：使用 cmd 包装脚本
javaagentcli.cmd

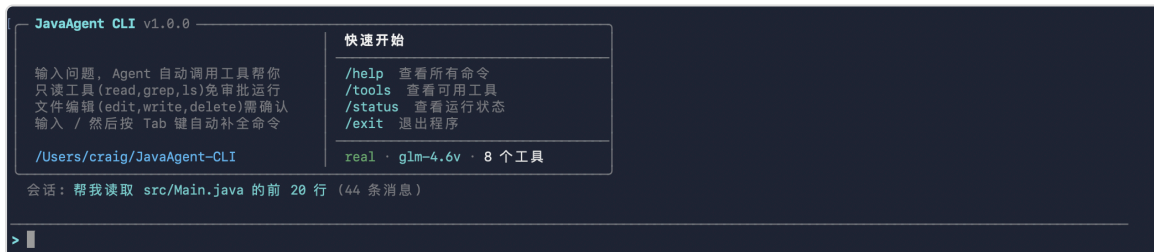
# 方式二：直接运行 JAR
java --enable-native-access=ALL-UNNAMED -jar target\javaagent-cli-1.0.0.jar
```

模拟模式（无需 API Key）

```
java --enable-native-access=ALL-UNNAMED -jar target/javaagent-cli-1.0.0.jar --mock
```

成功界面

启动成功后将看到如下界面：



```
JavaAgent CLI v1.0.0

输入问题，Agent 自动调用工具帮你
只读工具(read,grep,ls)免审批运行
文件编辑(edit,write,delete)需确认
输入 / 然后按 Tab 键自动补全命令

/Users/craig/JavaAgent-CLI

快速开始
/help 查看所有命令
/tools 查看可用工具
/status 查看运行状态
/exit 退出程序

real · glm-4.6v · 8 个工具

会话：帮我读取 src/Main.java 的前 20 行 (44 条消息)

>
```

五、配置说明

运行时行为通过 `config.properties` 文件控制。程序会按以下顺序查找配置文件：

1. 项目根目录（与 JAR 同级）
2. 当前工作目录
3. `~/.javaagent-cli/config.properties`

5.1 完整配置项

```
# ===== 基础配置 =====
agent.mock_mode=false           # 模拟模式开关
agent.api_key=                   # OpenAI 兼容 API Key
agent.base_url=                  # API 基础地址（根据使用的模型服务商填写）
agent.model=gpt-4o              # 模型名称

# ===== 行为配置 =====
agent.max_iterations=12         # 每轮最大工具调用次数
agent.bash_enabled=false        # 是否启用 bash 工具
agent.stream_responses=true     # 是否启用流式输出
agent.effort=high               # 推理深度（low/high/xhigh/max/ultra）

# ===== 上下文配置 =====
agent.max_tokens=200000         # 上下文窗口大小
agent.auto_compact_threshold=0.8 # 自动压缩阈值

# ===== 安全配置 =====
agent.rate_limit_qps=10         # API 速率限制（QPS）
agent.auto_save=true            # 自动保存会话
agent.approval_cache=true       # 审批缓存开关
agent.allow_external_paths=false # 是否允许访问工作区外路径

# ===== 自定义 =====
agent.custom_system_prompt=     # 自定义系统提示词（留空使用默认）
```

5.2 配置示例

```
# 模拟模式（开发/演示用）
agent.mock_mode=true

# 真实模式
agent.mock_mode=false
agent.api_key=sk-your-api-key-here
agent.base_url=https://api.deepseek.com/v1
agent.model=deepseek-chat
agent.bash_enabled=true
```

提示：配置修改后可在 CLI 中使用 `/reload` 命令热重载，无需重启程序。

六、使用指南

6.1 基本交互

启动程序后，在提示符处输入自然语言指令即可：

```
JavaAgent CLI v1.0.0 | mock-model | effort: high
```

```
> 帮我读取 README.md 文件
```

```
> 在 src/ 目录下搜索所有包含 "TODO" 的行
```

6.2 常用斜杠命令

命令	说明
<code>/help</code>	显示帮助信息
<code>/clear</code>	清除当前对话
<code>/context</code>	查看 token 使用情况
<code>/effort [level]</code>	设置推理深度 (low/high/xhigh/max/ultra)
<code>/mock</code>	切换到模拟模式
<code>/real</code>	切换到真实模式
<code>/model [name]</code>	查看或切换模型
<code>/prompt show set reset</code>	管理系统提示词
<code>/sessions</code>	列出历史会话
<code>/reload</code>	重新加载配置文件
<code>/stats</code>	查看工具调用统计
<code>/version</code>	查看版本信息

6.3 推理深度说明

级别	特点	适用场景
low	简短直接，跳过解释	快速问答、简单操作
high	逐步推理，详细解释	日常编程任务（默认）
xhigh	深度分析，多角度探索	复杂问题、架构设计
max	最大深度，考虑所有边界	关键代码审查
ultra	极限推理，穷举一切可能	极其复杂的系统设计

七、内置工具

工具名	别名	功能	参数
<code>read_file</code>	cat	读取文件内容	path, limit, offset
<code>write_file</code>	write	写入文件（支持追加）	path, content, append
<code>edit</code>	str_replace	精确字符串替换	path, old_string, new_string, replace_all
<code>delete_file</code>	rm	删除文件	path
<code>grep</code>	search, find	正则搜索文本	pattern, path, case_sensitive
<code>list_directory</code>	ls	列出目录内容	path, recursive, limit
<code>bash</code>	shell, exec	执行 Shell 命令	command
<code>network</code>	http	发送 HTTP 请求	method, url, body

八、斜杠命令

8.1 命令列表

在 CLI 提示符处输入 `/` 可查看所有可用命令并自动补全。

8.2 上下文管理

使用 `/context` 命令查看当前 token 使用情况，包括：

- 系统提示词占用的 token
- 工具定义占用的 token
- 对话消息占用的 token
- 剩余可用空间

当 token 使用超过阈值（默认 80%）时，系统会自动压缩对话历史。

8.3 会话管理

- `/sessions` — 列出所有历史会话
- `/clear` — 开始新会话
- 程序退出时自动保存当前会话
- 下次启动自动加载上次会话

九、安全机制

9.1 工作区隔离

文件操作默认限制在工作区内。访问工作区外的路径会被拒绝，除非在配置中明确启用 `agent.allow_external_paths=true`。

9.2 危险命令检测

BashTool 内置了 20 种危险命令模式检测：

类别	检测模式
文件系统破坏	<code>rm -rf /</code> 、 <code>rm -rf ~</code> 、 <code>chmod -R 777 /</code>
磁盘破坏	<code>mkfs</code> 、 <code>dd if=...of=/dev/</code>
系统破坏	<code>shutdown</code> 、 <code>:(){ : & }::</code>
安全攻击	反弹 shell、 <code>curl bash</code> 、 <code>kill -9 1</code>
数据泄露	<code>curl -d @/etc/passwd</code> 、 <code>shred</code>

9.3 敏感信息脱敏

工具执行结果在存储到会话文件前，会自动脱敏以下内容：

- API Key (sk-xxx 格式)
- Bearer Token
- agent.api_key 配置值

9.4 连续失败熔断

同一工具连续失败 3 次后，Agent 自动停止并提示，防止无限循环。

十、插件扩展

10.1 SPI 扩展机制

JavaAgent-CLI 支持通过 Java SPI (ServiceLoader) 机制扩展新工具：

1. 实现 `ToolProvider` 接口
2. 在 `META-INF/services/com.javagent.tools.ToolProvider` 中注册
3. 将 JAR 放入 classpath
4. 程序启动时自动发现并注册新工具

10.2 ToolProvider 接口

```
public interface ToolProvider {  
    /** 返回工具定义列表 */  
    List<Tool> provideTools();  
}
```

十一、项目结构

```
JavaAgent-CLI/
├── src/main/java/com/javagent/
│   ├── JavaAgentCLI.java          # CLI 入口, REPL 循环
│   ├── BannerPrinter.java         # 启动横幅 + 状态栏
│   ├── SlashCommandCompleter.java # 斜杠命令补全
│   ├── core/
│   │   ├── Agent.java            # ReAct 循环引擎
│   │   ├── Config.java           # 配置管理
│   │   ├── ConversationManager.java # 会话管理
│   │   ├── ApprovalManager.java  # 权限审批
│   │   └── ...
│   ├── model/
│   │   ├── ModelClient.java       # 模型客户端接口
│   │   ├── OpenAiCompatibleModelClient.java
│   │   ├── MockModelClient.java   # 模拟客户端
│   │   └── ...
│   ├── tools/
│   │   ├── Tool.java             # 工具接口
│   │   ├── ToolRegistry.java      # 工具注册中心
│   │   ├── ReadFileTool.java      # 读取文件
│   │   ├── EditTool.java          # 精确编辑
│   │   ├── BashTool.java          # Shell 执行
│   │   └── ...
│   └── util/
│       ├── Terminal.java          # ANSI 终端渲染
│       ├── TokenCounter.java       # Token 计数
│       ├── MarkdownRenderer.java   # Markdown 渲染
│       └── ...
├── src/test/java/                 # 66 个测试用例
├── pom.xml                       # Maven 配置
├── javaagentcli                  # macOS/Linux 启动脚本
├── javaagentcli.cmd              # Windows 启动脚本
└── config.properties             # 配置文件 (可选)
```

十二、常见问题

Q1: 启动时报 "Module enable-native-access" 错误

需要添加 JVM 参数: `--enable-native-access=ALL-UNNAMED`。使用提供的包装脚本可自动添加此参数。

Q2: 如何切换 AI 模型?

修改 `config.properties` 中的 `agent.model` 和 `agent.base_url`, 然后在 CLI 中执行 `/reload`

Q3: bash 工具默认禁用，如何开启？

```
在 config.properties 中设置：  
agent.bash_enabled=true
```

或使用 `/reload` 热重载。

Q4: 如何查看 token 使用情况？

输入 `/context` 命令，会显示彩色的 token 使用分布图。

Q5: 会话数据保存在哪里？

```
~/.javaagent-cli/sessions/
```

格式为 JSON。

Q6: 如何自定义系统提示词？

```
/prompt set <文本>      # 设置自定义提示词  
/prompt reset           # 恢复默认
```